

PERSONAL STUDY GUIDE · MAY 2026

SATM — How It Works, Start to Finish

Everything you need to know to explain the project confidently. 7 parts, plain language, no fluff.

[Part 1 — What & Why](#)[Part 2 — Tech Stack](#)[Part 3 — Auth](#)[Part 4 — NLP](#)[Part 5 — ML Pipeline](#)[Part 6 — API](#)[Part 7 — Deployment](#)

1

What is SATM and Why Does It Exist?

The problem, the solution, and the user value

The Problem

Students always have a pile of tasks — homework, exams, quizzes, projects. Three questions are always hard to answer:

- What type of task is this?
- How important and urgent is it?
- How long will it take?

They usually guess. They often get the priority wrong. Important things get left late.

The Solution

You type a task in plain English — exactly how you'd say it to a friend: "*finish bio hw ASAP*" or "*study for the databases exam in 3 days*". The AI reads the sentence and gives you back:

- **Category** — Homework, Exam, Quiz, or Project
- **Importance** — High, Medium, or Low
- **Estimated time** — how many hours it'll take
- **Priority score** — a 0–100 number combining urgency and importance
- **Daily schedule** — auto-generated 45-minute study sessions

Why AI and not just dropdowns?

Because typing is natural and fast. And because the AI can read context — "ASAP" means it's urgent, "bio" means biology, "hw" means homework. A simple form can't do that.

2

The Tech Stack

What tools we used and why

Think of the project as three layers:

Layer 1 — Frontend (what the user sees)

One file: **SATM.html**. Pure HTML, CSS, and JavaScript — no frameworks. A framework (like React or Vue) is a large pre-built system that changes how you write your entire frontend. We didn't need one — our app has one page and some dynamic updates. A framework would be overkill. Plain JS is simpler, faster to build, and easier to explain.

Layer 2 — Backend (the brain)

Flask — a Python library (just like NumPy or pandas, installed with pip). What makes it special is what it does: it turns your Python file into a web server. When you run `python app.py`, Flask starts listening for requests from the browser. You tell it: "if someone visits `/api/login`, run this function." That's it. Flask is not a separate language or software — it's a Python library that handles internet communication.

Layer 3 — Database (memory)

SQLite is a database engine — software that stores data in a structured file on disk (*satm.db*). It understands SQL (the language for databases). It was created independently but is bundled inside every Python installation, so no separate install is needed.

Flask-SQLAlchemy is a translator — instead of writing raw SQL commands, you write Python and it converts it to SQL behind the scenes.

Simple analogy: SQLite is like a filing cabinet. SQL is the language you use to ask for files. Flask-SQLAlchemy is the assistant who speaks both Python and SQL so you never have to leave Python.

The AI Layer

- **scikit-learn** — all ML models (Random Forest, SVM, Gradient Boosting)
- **NLTK** — text preprocessing (tokenization, lemmatization, stopwords)
- **scipy** — efficient math for combining feature matrices

Layer	Technology	Role
Frontend	HTML + CSS + JS	What the user sees and interacts with
Backend	Python + Flask	Processes requests, runs the AI, talks to DB
Database	SQLite + SQLAlchemy	Stores users and tasks permanently
AI / NLP	scikit-learn + NLTK	Understands and classifies tasks
Deployment	Render + gunicorn	Runs the app live, 24/7

3

Authentication

How register, login, and sessions work

Registration Flow

- 1 **User submits email + password** — the frontend validates format and password strength live before even sending anything
- 2 **Flask checks for duplicate email** — if it already exists, returns error: "account already exists"
- 3 **Password is hashed** — using Werkzeug. Hashing converts the password into a random-looking irreversible string. The real password is never stored anywhere.
- 4 **User saved to database** — email + hashed password written to the users table
- 5 **Session created** — Flask puts the user's ID in an encrypted cookie in the browser. This is how it remembers you're logged in.

Login Flow

Flask looks up the email, hashes the password the user typed, and compares it to the stored hash. If they match → session created. If not → "invalid email or password." The real password is never compared directly — only the hashes are compared.

What is a Session?

A session is a small encrypted cookie stored in the user's browser. Every request the browser sends includes this cookie. Flask reads it, finds the user ID inside, and knows who is making the request. When you log out, Flask clears the cookie — you become a stranger again.

@require_auth

Every protected route has this decorator above it. It's a security guard — before the function even runs, it checks: is there a valid session? If not, it returns a 401 error immediately and the function never executes.

4

NLP Preprocessing

How raw text gets turned into numbers the AI can read

Before any model touches your task, the raw text goes through 7 steps. Think of it as translating messy human language into something a machine understands.

Step 1 — Lowercasing

```
"Finish BIO HW ASAP" → "finish bio hw asap"
```

Step 2 — Punctuation & emoji removal

```
"study for exam!!! 🤔" → "study for exam"
```

Step 3 — Abbreviation expansion

```
hw → homework    bio → biology    ppt → presentation    idk → unknown
```

We built a small dictionary of common student abbreviations. The model was trained on full words, so we expand before feeding in.

Step 4 — Tokenization

```
"finish biology homework asap" → ["finish", "biology", "homework", "asap"]
```

The sentence is split into individual words. Each word is called a token. Done by NLTK's `word_tokenize`.

Step 5 — Stopword removal

```
["finish", "the", "biology", "homework", "by", "tonight"]  
→ ["finish", "biology", "homework", "tonight"]
```

Stopwords are common words with no real meaning — "the", "is", "a", "by", "for". NLTK has a built-in list. We remove them because they add noise without helping the model.

Step 6 — Lemmatization

```
studying → study    exams → exam    finished → finish
```

Every word is reduced to its root form. "studying" and "study" mean the same thing — lemmatization makes the model treat them as one word. Done by NLTK's `WordNetLemmatizer`.

Step 7 — TF-IDF Vectorization

The final step. The model can't work with words — it only understands numbers. TF-IDF converts the cleaned word list into a long vector of numbers, one per word in the vocabulary. Each number represents how important that word is for this task compared to all training tasks.

```
["finish", "biology", "homework"] → [0, 0, 0.43, 0, 0.71, 0, 0.52, 0, ...]
```

Most numbers are zero (words not present). The non-zero ones are the words that appeared. This is called a **sparse vector** — and this is what gets fed into the ML models.

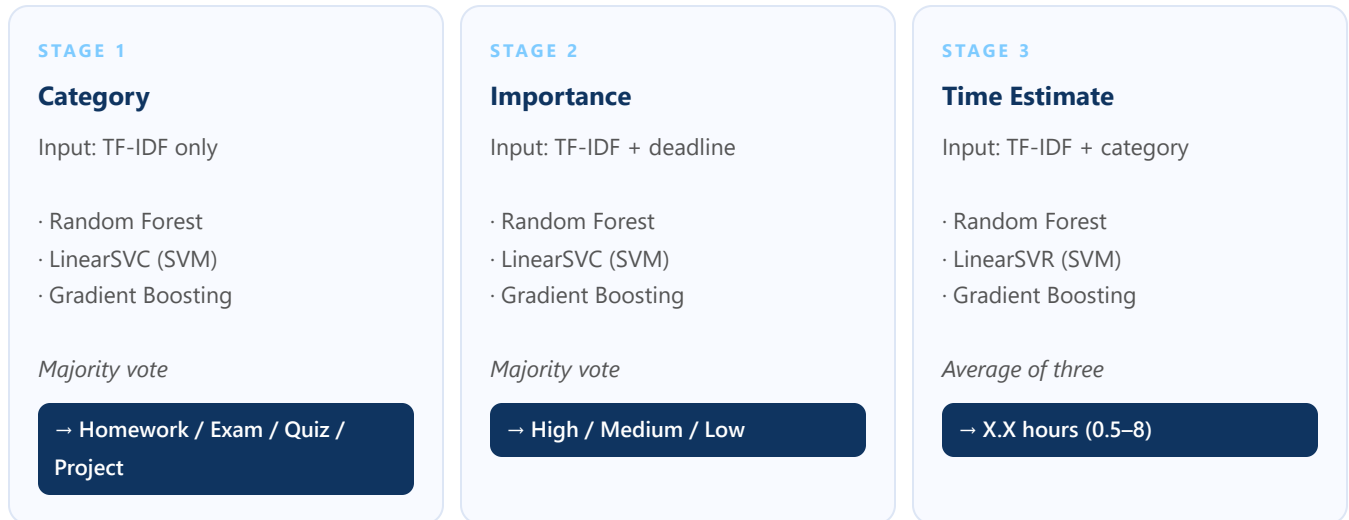
One sentence summary: Raw messy text → cleaned → split into words → noise removed → roots extracted → converted to numbers → ready for AI.

5

The ML Pipeline — Core of the Project

3 stages · 9 models · ensemble voting

After preprocessing, you have a vector of numbers. Now three prediction stages run one after another — each feeds into the next.



Why does each stage get different inputs?

- **Stage 1 (Category)** — text alone is enough. "exam", "homework", "quiz" are obvious signals.
- **Stage 2 (Importance)** — needs the deadline too. The same homework with 1 day left is High; with 14 days it might be Medium. Text alone can't decide.
- **Stage 3 (Time)** — needs the category. An exam takes longer to prepare for than a quiz. Category gives context about workload.

Classification vs Regression

Classification = pick from a fixed list (Category and Importance). Winner decided by majority vote across 3 models.

Regression = predict a continuous number (Time). The 3 models each give a number and we average them: RF: 2.0h + SVR: 1.8h + GB: 2.1h → average = **2.0h**.

The 3 Models — How Each One Thinks

Model	How it works
Random Forest	Builds hundreds of decision trees, each trained on a random slice of data. Takes the majority vote across all trees. Resistant to overfitting.
SVM (Support Vector Machine)	Finds the mathematical boundary that separates classes with the maximum gap between them. Historically the strongest model for text classification.
Gradient Boosting	Builds trees sequentially — each tree corrects the mistakes of the previous one. Repeats 150 times. Captures complex patterns the others might miss.

Why 3 models instead of 1? — The answer for your professor

"A single model can be confidently wrong. By running three independent models that each learn differently, we reduce that risk. Random Forest averages hundreds of trees. SVM finds the widest mathematical boundary. Gradient Boosting fixes its own mistakes iteratively. They fail in different ways — so when they agree, we can trust the result. When they disagree, we warn the user with a badge."

Model Accuracy

Prediction	Random Forest	SVM	Gradient Boosting
Category	99.5%	99.5%	99.0%
Importance	100%	99.5%	100%
Time (MAE)	~57 min	~58 min	~57 min

Time accuracy is lower because it's a regression problem on a continuous scale — and the same task genuinely takes different people different amounts of time. $R^2 \approx 0.50$ means the models explain about half the real variation, which is reasonable given 1,000 training samples and inherent human variability.

6

The API

How the frontend and backend talk to each other

API = Application Programming Interface. It's the agreed set of rules for how two pieces of software send messages to each other. The frontend (HTML page) and backend (Flask) are separate — they communicate only through HTTP requests.

Every request has three things

- **Method** — GET (fetch something), POST (send something), DELETE (remove), PATCH (update)
- **URL** — which specific thing you're asking about
- **Body** — the data you're sending (for POST/PATCH requests)

All API Endpoints

Method	URL	What it does
POST	/api/register	Create new account
POST	/api/login	Sign in
POST	/api/logout	Sign out
GET	/api/me	Check who is currently logged in
POST	/api/predict	Run the full ML pipeline on a task
GET	/api/tasks	Get all saved tasks for this user
POST	/api/tasks	Save a task to the database
DELETE	/api/tasks/<id>	Delete a specific task
PATCH	/api/tasks/<id>	Update a task (status, importance, etc.)

Real example — clicking Analyze

1. You type "finish bio hw ASAP", deadline = 1, click Analyze
2. Frontend sends: POST /api/predict with body `{ text: "finish bio hw ASAP", deadline: 1 }`
3. Flask runs the full NLP + ML pipeline
4. Flask responds with:

```
{ "category": "Homework", "cat_agree": 3, "importance": "high",  
  "imp_agree": 3, "time_est": 1.5, "time_spread": 0.2 }
```

5. Frontend reads that and updates the page — fills dropdowns, shows agreement badges, calculates priority score.

Restaurant analogy: The frontend is the customer. The backend is the kitchen. The API is the menu — a fixed list of things you can order. The customer never walks into the kitchen; they just place an order and wait for the result.

7

Deployment

How the app runs live for everyone

Why not just run locally?

When you run `python app.py` on your machine, the app works — but only for you, only on your computer, only while the terminal is open. The moment you close the laptop, it's gone. Deployment puts the app on a server that stays on 24/7.

What is Render?

Render is a cloud hosting platform — a company that rents you a computer in a data center that never turns off. You give them your code, they run it. We use the free tier — no payment, but with two limitations:

- The server **sleeps** after 15 minutes of no traffic (first visit after sleep takes ~30 seconds to wake up)
- The filesystem is **ephemeral** — the SQLite database resets on every redeploy (planned fix: migrate to PostgreSQL)

How code gets to Render

Your machine → GitHub → Render (auto-redeploys on every push)

This is called continuous deployment — you never manually upload anything to Render. Push code, Render picks it up automatically.

Three key files for deployment

- **Procfile** — tells Render how to start the app: `web: gunicorn app:app`
- **requirements.txt** — tells Render which Python libraries to install (Flask, scikit-learn, nltk, etc.)
- **render.yaml** — configuration file: Python version, environment variables, service type

Why gunicorn and not just Flask?

Flask's built-in server handles one request at a time. If two users click Analyze simultaneously, one waits. Gunicorn runs multiple workers in parallel — it's a production-grade web server and the standard way to deploy Flask apps.

Environment Variables

Two sensitive values are never stored in the code or on GitHub:

- **SECRET_KEY** — encrypts session cookies. If someone knows this, they can forge logins.
- **DATABASE_URL** — the database connection string

These are set in Render's dashboard. The code reads them with `os.environ.get('SECRET_KEY')`.

Full request journey on the live site

```

User's browser
  ↓ HTTPS request
Render server
  ↓
gunicorn (receives and distributes the request)
  ↓
Flask – app.py runs, processes the request
  ↓
ML models + SQLite database
  ↓
Response back to the browser

```

FULL PROJECT SUMMARY

Part	Topic	Key point
1	What & Why	AI that classifies, prioritizes, and schedules academic tasks from plain text
2	Tech Stack	Flask (Python web server) + SQLite (database) + HTML/JS (frontend) + scikit-learn (AI)
3	Auth	Passwords hashed with Werkzeug, sessions via encrypted browser cookie, @require_auth guard
4	NLP	Clean → tokenize → remove stopwords → lemmatize → TF-IDF vector of numbers
5	ML Pipeline	3 stages (category → importance → time), each with 3 models, majority vote or average
6	API	Frontend sends HTTP requests to Flask endpoints, gets JSON back, updates the page
7	Deployment	Code pushed to GitHub → Render auto-redeploys → gunicorn serves requests live